

Combinational circuits using "assign" statement

```
module example(a,b,c);
input a,b;
output c;
```

```
assign a = 1'b1, b = 2'b0;
```

```
assign c = a | b; // a OR b = a + b
```

```
initial begin
```

```
$monitor ("c = ", c);
```

```
end
```

```
endmodule
```

Ans. c = 1

Explicitly define X

```
module example(a,b,c);
```

```
input a,b;
```

```
output reg c;
```

```
assign a = 1'b1, b = 2'b0;
```

```
initial begin
```

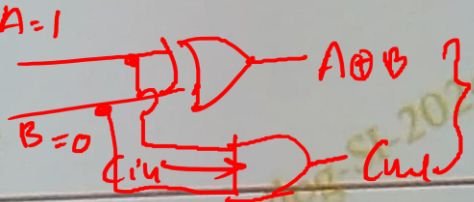
```
c = a | b;
```

```
$monitor ("c = ", c);
```

```
end
```

```
endmodule
```

Ans. c = 1



Java ✓

Public {

{String my .EJ}

int pos, element;

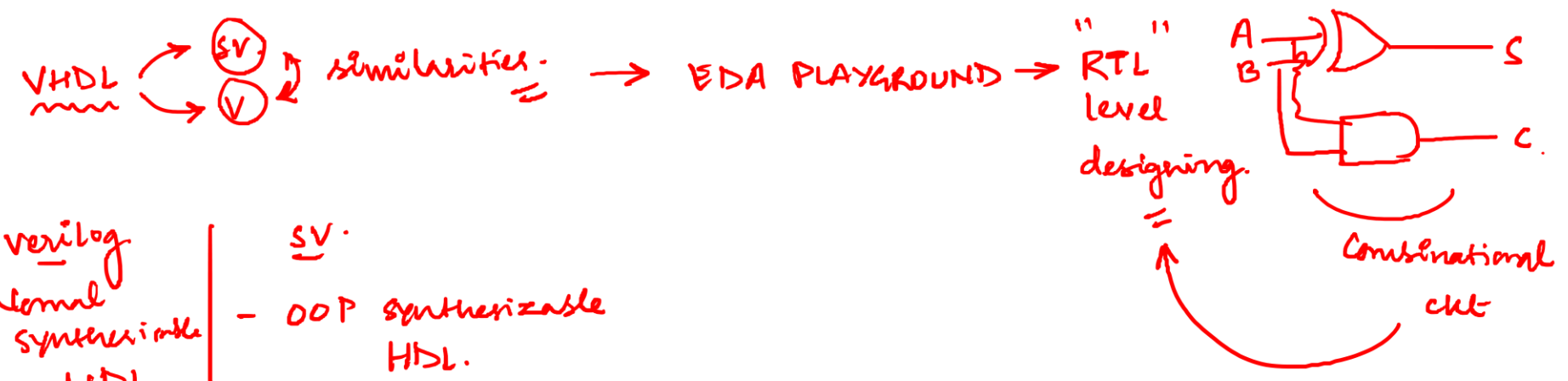
HA = 1 + 0 = 1

<https://www.edaplayground.com/x/7YJ6>

Notes:

- Wire can't be initialized within the initial block or an always block
- Wire should be initialized outside the initial block using the assign statement
- Register has to be initialize within an initial block or an always block

-- ① VHDL ✓ → strong-typed language → Errors ↑ ; Debugging - difficult
 case sensitive
 Verification level is low.
 -- System Verilog ✓ → OOPS - class - interface, virtual interface, etc.
 -- Verilog ✓



Verilog	SV.
- Normal synthesizable HDL.	- OOP synthesizable HDL.
	- class → obj → interface
	→ virtual interface.

module moduleName
 tb { module_HATb | design.

2-38-①
 39-102-②
 Parameters
 *
 class
 constructor...
 (SV)

module initial_Example ;

Module Name

reg a,b,c ; Inputs declaration.

wire sum,cout; o/p declaration.

[initial begin]

[c = 1'b0]; Initialised @ t=0s → BINARY

#15 c = 1'b1;

Block #5 c = 1'b0 ;

#5 c = 1'b1 ;

#5 c = 1'b0 ;

[end]

[initial begin

#5 a = 1'b1 ; b = 1'b1 ;

#5 b = 1'b0 ;

end

initial begin

\$monitor("time = %0d , a = %b , b = %b , c = %b" , \$time , a , b , c) ;

#25 \$finish ;

end

endmodule

No. of Bits representation

BIT

1

6

0

→

value

eg :

2'b00 → 00 : 0 ✓

2'b01 → 01 : 1 ✓

2'b10 → 10 : 2 ✓

2'b11 → 11 : 3 ✓

PARALLEL EXEC.

time = 0 , a = x , b = x , c = 0

time = 5 , a = 1 , b = 1 , c = 0

time = 10 , a = 1 , b = 0 , c = 0

time = 15 , a = 1 , b = 0 , c = 1

time = 20 , a = 1 , b = 0 , c = 0

testbench.sv:21: \$finish called at 25 (1s)

time = 25 , a = 1 , b = 0 , c = 1

“ Perspective of the Number of Initial Begin Blocks...

In BLOCKING statements: Execution happens Sequentially According to increasing order of time...

0 → 5 → 10 → 15 → so on.”

4'b0000 → 0000

4'b1111

4'h00 → [0 → 15].

6'b000000

DELAYS → Block wise → Initial-Begin / Fork-Join
 statements → Blocking.
 Non-Blocking.

```
module initial_Example ;
```

```
    reg a,b,c ;
    wire sum,cout;
    initial begin
```

@ t=0s → c = 1'b0 ; → a ?
 → #15 c = 1'b1 ; → b ?
 → #5 c = 1'b0 ;
 #5 c = 1'b1 ;
 #5 c = 1'b0 ;

a = x b = x c = 0

```
end
```

initial begin
 → #5 a = 1'b1 ; b = 1'b1 ;
 #5 b = 1'b0 ;
 end

// 5 a=1 , b=1 , c=0
 // 10 a=1 , b=0 , c=0

```
    initial begin
```

```
        $monitor("time = %0d , a = %b , b = %b , c = %b" , $time , a , b , c) ;
        #25 $finish ;
```

%h → HEXADECIMAL.

BINARY.

```
    end
endmodule
```

```
module initial_Example ;
```

```
    reg a,b,c ;
    wire sum,cout;
    initial begin
```

1st
Initial
Block
=

t = 0s → c = 1'b0 ;
 5s → #15 c = 1'b1 ; X ✓
 +10s → #5 c = 1'b0 ;
 15s → #5 c = 1'b1 ;
 15s → #5 c = 1'b0 ; ✓

```
    end
```

2nd
Initial
Block
=

```
    initial begin
        5s → #5 a = 1'b1 ; b = 1'b1 ;
        10s → #5 b = 1'b0 ;
    end
```

```
    initial begin
```

```
        $monitor("time = %",
```

```
    end
endmodule
```

time: 0s → a = x b = x c = 0 ✓
 time: 5s → a = 1 b = 1 c = 0 ✓
 time: 10s → a = 1 b = 0 c = 0 ✓
 time: 15s → a = 1 b = 0 c = 1 ✓
 time: 20s → a = 1 b = 0 c = 0
 → \$finish 25s *

time 25s

```
        [2024-08-28 17:53:17 UTC] iverilog -Wall -g2012 design.sv testbench.sv && unbuffer vvp a.out
```

```
time = 0 , a = x , b = x , c = 0
time = 5 , a = 1 , b = 1 , c = 0
time = 10 , a = 1 , b = 0 , c = 0
time = 15 , a = 1 , b = 0 , c = 1
time = 20 , a = 1 , b = 0 , c = 0
testbench.sv:23: $finish called at 25 (1s)
time = 25 , a = 1 , b = 0 , c = 1
```

Done

Block statements

➤ The Verilog HDL has two types of blocks.

- Sequential (begin-end blocks) → SEQUENTIAL CNTS

- All statements within sequential blocks are executed **in the order in which they are written.**
- If a timing control statement appears within a block, then the **next statement will be executed after that delay**

- Parallel (fork-join blocks)

- All statements within parallel blocks are **executed at the same time.**
- The execution of the **next statement will not be delayed** even if the previous statement **contains a timing control statement.**

→ "COUNTERS" → "NON-BLOCKING Assignment" Statements ($<=>$).

(begin-end) vs (fork-join)

module main;

initial **begin**

#20 \$display("time1 = %0d", \$time);

#10 \$display("time2 = %0d", \$time);

#5 \$display("time3 = %0d", \$time);

end

endmodule

<https://www.edaplayground.com/x/Rgqg>

ONE SINGLE PROGRAM.

time1 = 20

time2 = 30

time3 = 35

time-stamps will be getting added !!

module main;

initial **fork**

#20 \$display("time1 = %0d", \$time);

#10 \$display("time2 = %0d", \$time);

#5 \$display("time3 = %0d", \$time);

join

endmodule

<https://www.edaplayground.com/x/X6cY>

ANOTHER PROGRAM.

time3 = 5

time2 = 10

time1 = 20

LESS TIME; FIRST EXECUTION !!

Execution will start according to increasing order of assigned time.

No add of time-stamps.

0 → 2 → 5 → 7 → 9

(begin-end) and (fork-join) in one program

module main ;

initial **begin** ✓

#20 \$display("BE: time1 = %0d", \$time);

#10 \$display("BE: time2 = %0d", \$time);

#5 \$display("BE: time3 = %0d", \$time);

end

initial **fork** ✓

#20 \$display("FJ: time1 = %0d", \$time);

#10 \$display("FJ: time2 = %0d", \$time);

#5 \$display("FJ: time3 = %0d", \$time);

join

endmodule

When BE ties with FJ, FJ is more prior than BE.

FJ >> BE !!

FJ: time3 = 5 ✓

FJ: time2 = 10 ✓

BE: time1 = 20 *

FJ: time1 = 20 *

BE: time2 = 30 ✓

BE: time3 = 35 ✓

FJ will be executed FIRST.

all to the time !!

Increasing order of time-stamps.
= 00

(begin-end) and (fork-join) in one program (Question)

```
module main ;
```

```
initial fork
```

```
#20 $display("FJ: time1 = %0d", $time);
```

```
#10 $display("FJ: time2 = %0d", $time);
```

```
#5 $display("FJ: time3 = %0d", $time);
```

```
join
```

```
initial begin
```

```
#20 $display("BE: time1 = %0d", $time);
```

```
#10 $display("BE: time2 = %0d", $time);
```

```
#5 $display("BE: time3 = %0d", $time);
```

```
end
```

```
endmodule
```

Fork join is always More prior than Begin
End Because (increasing order of time stamps).

FJ: time3 = 5

FJ: time2 = 10

FJ: time1 = 20

BE: time1 = 20

BE: time2 = 30

BE: time3 = 35

Execution will
Be in the
order of inc.
time stamps.

(begin-end) and (fork-join) in one program (Question)

```
module main ;  
initial begin  
    #20 $display("BE: time1 = %0d", $time );  
    #10 $display("BE: time2 = %0d", $time );  
    #5 $display("BE: time3 = %0d", $time );  
end
```

```
initial fork  
    #22 $display("FJ: time1 = %0d", $time );  
    #10 $display("FJ: time2 = %0d", $time );  
    #5 $display("FJ: time3 = %0d", $time );  
join
```

```
endmodule
```

FJ: time3 = 5

FJ: time2 = 10

BE: time1 = 20

FJ: time1 = 22

BE: time2 = 30

BE: time3 = 35

} Fork join Execution
will be first prior.

→ then begin end. (Acc. to increasing
time stamps)

→ followed By Fork join
[acc. to time].

→ According to time.

(fork-join) inside a (begin-end)

```
module main ;
```

```
initial begin
```

```
  #10 $display("BEFORE fork: time = %0d", $time);
```

```
  fork
```

```
    $display("Inside fork: time = %0d", $time);
```

```
    #20 $display("time1 = %0d", $time);
```

```
    #10 $display("time2 = %0d", $time);
```

```
    #5 $display("time3 = %0d", $time);
```

```
  join
```

```
    $display("Outside the main fork: time = %0d", $time);
```

```
  end
```

```
endmodule
```

BEFORE fork: time = 10

Inside fork: time = 10

time3 = 15

time2 = 20

time1 = 30 - last time

Outside the main fork: time = 30

Executed first

Executed second.

All statements in this fork-join block starts at same time executed in **Parallel**

Executed third

starting from here...
PARALLEL EXECUTION

#nos.

Blocking Assignment (With delay)

```
module main ;  
integer a, b, c;
```

```
initial  
begin
```

```
a = 10; b = 20; c = 15;
```

```
end
```

```
initial begin
```

```
$monitor("time=%0d: a=%0d,  
b=%0d, c=%0d", $time, a, b, c);
```

```
a = #5 b + c;
```

```
b = #5 a + 5;
```

```
c = #5 a - 10;
```

```
end
```

```
endmodule
```

initialised values....

Inside initial block; time-stamps are always added.

Ans:

time=0: a=10, b=20, c=15

time=5: a=35, b=20, c=15

time=10: a=35, b=40, c=15

time=15: a=35, b=40, c=25

- The three lines will start execution one by one.
- The specified delay of 1st statement will block the execution of the 2nd statement for that much time

Non-Blocking Assignment: Ex-1 (without delay)

```
module main ;  
integer a, b, c;  
initial  
begin  
a = 10; b = 20; c = 15;  
end  
initial begin  
$monitor("time=%0d: a=%0d,  
b=%0d, c=%0d", $time, a, b, c);  
a <= b + c;  
b <= a + 5;  
c <= a - 10;  
end  
endmodule
```

- The initial value of **a=10** is considered
- a is not considered as **35**

$$a = b + c = 20 + 15 = 35$$

$$b = a + 5 = 10 + 5 = 15$$

$$c = a - 10 = 10 - 10 = 0$$

- All three line will start execution at **t=0**
- The initial values of a, b, c (**when the set of non blocking statements starts execution**) will be considered for execution of all the statements

Non-Blocking Assignment : Ex-2 (with delay)

```
module main ;  
integer a, b, c;  
initial  
begin  
a = 10; b = 20; c = 15;  
end  
initial begin  
$monitor("time=%0d: a=%0d,  
b=%0d, c=%0d", $time, a, b, c);  
a <= #5 b + c;  
b <= #10 a + 5;  
c <= #5 a - 10;  
end  
endmodule
```

@ t = 5s; As it is NON-BLOCKING Assignment- Statement...
Execution will be @ the common time @ @
Same time.

Ans:

- ① time=0: a=10, b=20, c=15
- ② time=5: a=35, b=20, c=0
- ③ time=10: a=35, b=15, c=0

- All three line will start execution at t=0
- But it will be assigned to LHS after specified delay
- The initial values of a, b, c (when the set of non blocking statements starts execution) will be considered for execution of all the statements

```

module main ;
  integer a , b , c ;
  initial begin
    a = 10 ; b = 20 ; c = 15 ;
  end
  initial begin
    $monitor("time = %0d , a = %0d , b = %0d , c = %0d " , $time , a , b , c) ;
    a <= b + c ;
    b <= a + 5 ;
    c <= a - 10 ;
  end
endmodule

```

If there will be "NO DELAYS", then in NON-BLOCKING Assignment statements,
 All execution will be @ a same time i.e. t = 0s.

time = 0 , a = 35 , b = 15 , c = 0

— o/p.

Non-Blocking Assignment : Ex-3

```
module main ;  
integer a, b, c;  
initial  
begin  
a = 10; b = 20; c = 15;  
end  
initial begin  
$monitor("time=%0d: a=%0d,  
b=%0d, c=%0d", $time, a, b, c);  
a <= #5 b + c;  
#10  
b <= a + 5;  
c <= #5 a - 10;  
end  
endmodule
```

- The initial value of **a** is **35** when these 2nd set of instructions are executed

Ans:

time=0: a=10, b=20, c=15
time=5: a=35, b=20, c=15
time=10: a=35, b=40, c=15
time=15: a=35, b=40, c=25

- Start execution at t=0
- b & c will start execution at t=10
- c is assigned after #5 unit of starting execution (i.e. 10+5 = 15)

Non-Blocking Assignment : Ex-4

```
module main ;  
integer a, b, c;  
initial  
begin  
a = 10; b = 20; c = 15;  
end  
initial begin  
$monitor("time=%0d: a=%0d,  
b=%0d, c=%0d", $time, a, b, c);  
a <= #5 b + c;  
#10  
a <= a + 5;  
c <= #5 a - 10;  
end  
endmodule
```

- The initial value of **a** is **35** when these 2nd set of instructions are executed

T=5: $a = b + c = 20 + 15 = 35$
T=10: $a = a + 5 = 35 + 5 = 40$
T=15: $c = a - 10 = 35 - 10 = 25$

Start execution at t=0

b & c will start execution at t=10

C is assigned after #5 unit of starting execution (i.e. $10 + 5 = 15$)

Ans:

time=0: a=10, b=20, c=15

time=5: a=35, b=20, c=15

time=10: a=40, b=20, c=15

time=15: a=40, b=20, c=25

Non-Blocking Assignment : Ex-5

```

module main ;
integer a, b, c;
initial
begin
a = 10; b = 20; c = 15;
end

```

→ @ t=0s

```

initial begin
$monitor("time=%0d: a=%0d,
b=%0d, c=%0d", $time, a, b, c);

```

```

a <= #15 b + c;

```

#10 : this time-stamp is common

```

a <= a + 5;

```

```

c <= #5 a - 10;

```

```

end

```

```

endmodule

```

T=15: $a = b + c = 20 + 15 = 35$

T=10: $a = a + 5 = 10 + 5 = 15$

Start execution at t=0

b & c will start execution at t=10

C is assigned after #5 unit of starting execution (i.e. $10 + 5 = 15$)

Ans:

time=0: a=10, b=20, c=15 ✓

time=10: a=15, b=20, c=15 ✓

time=15: a=35, b=20, c=10 ✓

to the below written statements as there is NO SEMI-COLON (;) in front of #10 and; it wasn't terminated; hence Control went to #10s first then #15 sec.

Because; it's Under Initial-begin-end and there time-stamps get Added !!

Non-Blocking Assignment : Ex-6

```
module main ;  
integer a, b, c;  
initial  
begin  
a = 10; b = 20; c = 15;  
end  
initial begin  
$monitor("time=%0d: a=%0d,  
b=%0d, c=%0d", $time, a, b, c);  
a <= #5 b + c;  
#2 a <= a + 5;  
c <= #5 a - 10;  
end  
endmodule
```

- The initial value of a is 10 when these 2nd set of instructions are executed

$$T=2: a = a + 5 = 10 + 5 = 15$$

$$T=5: a = b + c = 20 + 15 = 35$$

$$T=7: c = a - 10 = 10 - 10 = 0$$

Start execution at $t=0$

a & c will start execution at $t=2$

c is assigned after #5 unit of starting execution (i.e. $2+5=7$)

Ans:

time=0: a=10, b=20, c=15

time=2: a=15, b=20, c=15

time=5: a=35, b=20, c=15

time=7: a=35, b=20, c=0

Just like FORK-JOIN
Here, In NON-BLOCKING also
Execution will be in increasing
orders of time stamps.

CASE-01 : Initial Block ke andar for join likhenge; then timestamps will be Added.

CASE-02 : CASE-01 in NON-BLOCKING; time-stamps will NOT BE ADDED.

Non-Blocking Assignment : Question

@t=0s →

```
module main;  
integer a, b, c;  
initial  
begin  
a = 10; b = 20; c = 15;  
end  
initial begin  
$monitor("time=%0d: a=%0d,  
b=%0d, c=%0d", $time, a, b, c);  
a <= #5 b + c;  
c <= #4 a - 10;  
b <= #10 a + 5;  
end  
endmodule
```

@t=5s →
@t=4s →
@t=10s →

```
time=0: a=10, b=20, c=15  
time=4: a=10, b=20, c=0 ✓  
time=5: a=35, b=20, c=0  
time=10: a=35, b=15, c=0
```

```
module main;  
integer a, b, c;  
initial  
begin  
a = 10; b = 20; c = 15;  
end  
initial begin  
$monitor("time=%0d: a=%0d,  
b=%0d, c=%0d", $time, a, b, c);  
a <= #5 b + c;  
c <= #4 a - 10;  
#4  
b <= #10 a + 5;  
end  
endmodule
```

```
time=0: a=10, b=20, c=15  
time=4: a=10, b=20, c=0  
time=5: a=35, b=20, c=0  
time=14: a=35, b=15, c=0
```

```
module main;  
integer a, b, c;  
initial  
begin  
a = 10; b = 20; c = 15;  
end  
initial begin  
$monitor("time=%0d: a=%0d,  
b=%0d, c=%0d", $time, a, b, c);  
a <= #5 b + c;  
c <= #4 a - 10;  
#6  
b <= #10 a + 5;  
end  
endmodule
```

```
time=0: a=10, b=20, c=15  
time=4: a=10, b=20, c=0  
time=5: a=35, b=20, c=0  
time=16: a=35, b=40, c=0
```

(Non-Blocking Assignment in Begin-End) Vs (Blocking Assignment in fork-join)

```

module main ;
integer a, b, c;
initial
begin
a = 10; b = 20; c = 15;
end
initial begin
$monitor("time=%0d: a=%0d,
b=%0d, c=%0d", $time, a, b, c);
a <= #5 b + c;
c <= #4 a - 10;
b <= #10 a + 5;
end
endmodule
    
```

Question-1

time=0: a=10, b=20, c=15
time=4: a=10, b=20, c=0
time=5: a=35, b=20, c=0
time=10: a=35, b=15, c=0

```

module main ;
integer a, b, c;
initial
begin
a = 10; b = 20; c = 15;
end
initial fork
$monitor("time=%0d: a=%0d,
b=%0d, c=%0d", $time, a, b, c);
a = #5 b + c;
c = #4 a - 10;
b = #10 a + 5;
join
endmodule
    
```

time=0: a=10, b=20, c=15
time=4: a=10, b=20, c=0
time=5: a=35, b=20, c=0
time=10: a=35, b=15, c=0

@t=0s

@t=5s

@t=4s

@t=10s

SAME
AS

PREVIOUS

=

Swapping two variables : (Using blocking assignment)

// Using Blocking assignment

```
always @(posedge clk)
a = b;
always @(posedge clk)
b = a;
```

Wrong

- Either a=b will execute before b=a, or vice versa, depending which always block execute first
- Both registers will get the same value (either "a" or "b").
- Race condition occurs

```
always @(posedge clk)
begin
a = b;
b = a;
end
```

Wrong

- Both "a" and "b" will be getting the value previously stored in "b".

= Blocking.

```
always @(posedge clk)
begin
ta = a;
tb = b;
a = tb;
b = ta;
end
```

Correct

- Correct swapping will occur,
- but we need two temporary variables "ta" and "tb"

a ← b

temp-variable a = [a] } temp-a, a
temp-variable b = [b] } temp-b, b

Swapping two variables : (Using non-blocking assignment)

Correct way of swapping using non-blocking assignment

// Using Non-Blocking assignment

```
always @(posedge clk)
```

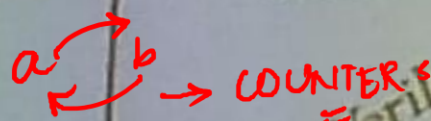
```
a <= b; ✓
```

```
always @(posedge clk)
```

```
b <= a; ✓
```

• Here the variables are correctly swapped.

• All RHS variables are read first, and assigned to LHS variables at the positive clock edge.

 COUNTER =

```
always @(posedge clk)
```

```
begin
```

```
a <= b;
```

```
b <= a;
```

```
end
```

• All RHS values (a,b) are read first,

• Those values are assigned to LHS variables at the positive clock edge.

• Swapped correctly

Swapping two variables : Blocking vs Non-Blocking

Blocking Assignment

Wrong

```
module main ;  
integer a, b, ta, tb;  
initial begin  
    a = 10; b = 20;  
    $monitor("a=%0d,  
b=%0d", a, b);  
    a = b;  
    b = a;  
end  
endmodule
```

a=20, b=20

Right

```
module main ;  
integer a, b, ta, tb;  
initial begin  
    a = 10; b = 20;  
    $monitor("a=%0d,  
b=%0d", a, b);  
    ta = a;  
    tb = b;  
    a = tb;  
    b = ta;  
end  
endmodule
```

a=20, b=10

Non-Blocking Assignment

Right

```
module main ;  
integer a, b;  
initial begin  
    a = 10; b = 20;  
    $monitor("a=%0d,  
b=%0d", a, b);  
    a <= b;  
    b <= a;  
end  
endmodule
```

a=20, b=10

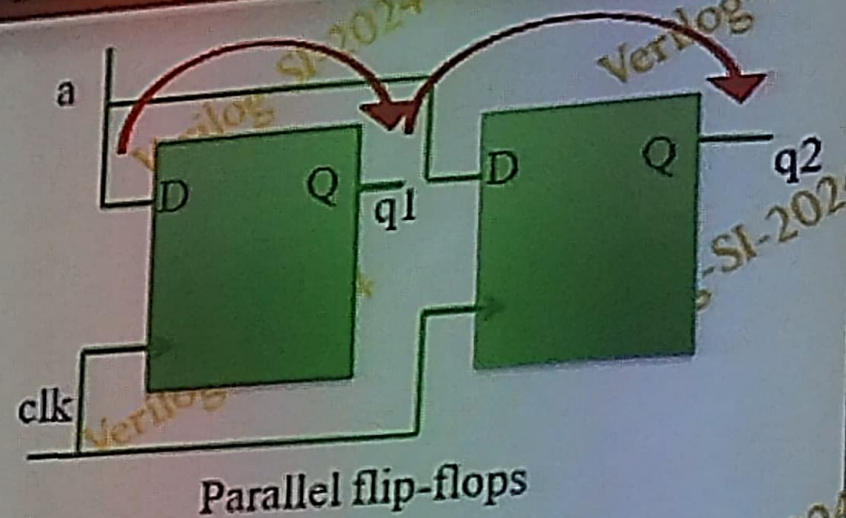
Both the RHS values are read first, then they are assigned to LHS

Blocking: Example

```
always @(posedge clock)
begin
  q1 = a;
  q2 = q1;
end
```

This is non blocking assignment.

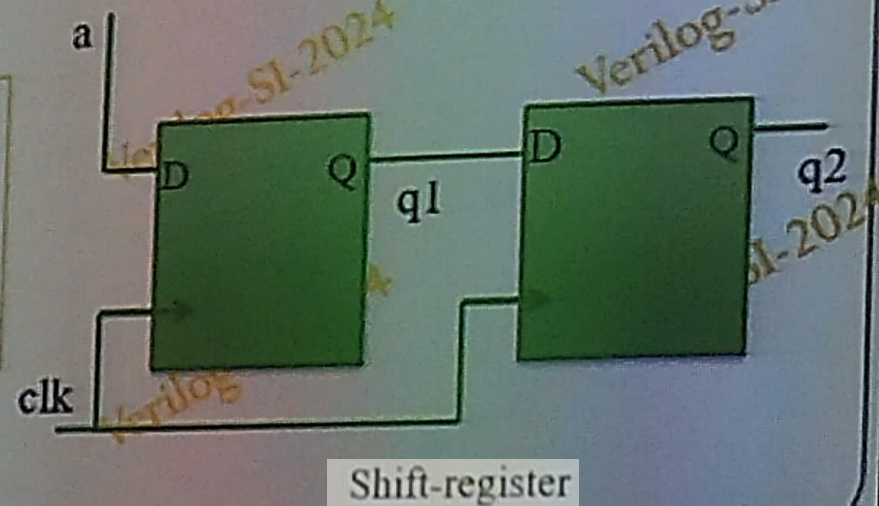
- q1 first takes the value of a
- Then q2 gets the value of q1 which is already equal to a



```
always @(posedge clock)
begin
  q2 = q1;
  q1 = a;
end
```

This is blocking assignment.

- q2 first takes the value of q1.
- Then q1 is assigned as a



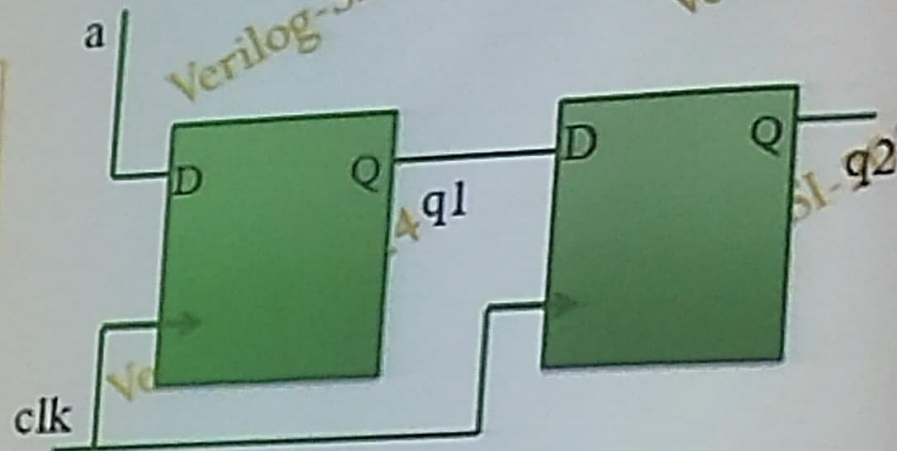
Non-Blocking: Example

```
always @(posedge clock)
begin
  q1 <= a;
  q2 <= q1;
end
```

- This is non blocking assignment.
- a and q1 are read first.
 - Then they are assigned to q1 and q2 respectively.
 - The initially stored value of q1 will be assigned to q2

```
always @(posedge clock)
begin
  q2 <= q1;
  q1 <= a;
end
```

- This will also synthesize the same circuit.
- The order of execution does not matter



Blocking: Example

design:sv

```
module shiftreg_4bit (clock, clear, A, E);  
    input clock, clear, A;  
    output reg E; reg B, C, D;  
    always @(posedge clock or negedge clear)  
    begin  
        if (clear)  
            B=0; C=0; D=0; E=0;  
        else  
            E=D;  
            D=C;  
            C=B;  
            B=A;  
            $display("[%0t], A=%b, B=%b, C=%b,  
            D=%b, E=%b", $time, A, B, C, D, E);  
        end  
    end  
endmodule
```

00:00
RESET

What circuit will the synthesis tool generate for the given designs?

It will be a **shift register** (It is written using blocking assignment)

Because it's Blocking

Initial values

A=1, B=0, C=0, D=0, E=0;

[7], A=1, B=1, C=0, D=0, E=0
[15], A=1, B=1, C=1, D=0, E=0
[25], A=1, B=1, C=1, D=1, E=0
[35], A=1, B=1, C=1, D=1, E=1
[45], A=1, B=1, C=1, D=1, E=1
[55], A=1, B=1, C=1, D=1, E=1

<https://www.edaplayground.com/x/7XDg>

```
module shiftreg_4bit_tb;  
    reg clock, clear, A;  
    wire E;  
    shiftreg_4bit sr(clock, clear, A, E);
```

always #5 clock = ~clock;

initial

begin

clock <= 0;

clear <= 1; A <= 1;

#7 clear = 0;

#200 \$finish;

end

endmodule

clock = 0

Non-Blocking: Example

```
module shiftreg_4bit (clock, clear, A, E);  
    input clock, clear, A;  
    output reg E; reg B, C, D;  
    always @(posedge clock or negedge clear)  
    begin  
        if (clear)  
            begin  
                B=0; C=0; D=0; E=0;  
            end  
        else  
            begin  
                B <= A;  
                C <= B;  
                D <= C;  
                E <= D;  
            end  
        $display("[%0t], A= %b, B= %b, C= %b,  
                D= %b, E= %b", $time, A, B, C, D, E);  
    end  
end  
endmodule
```

What circuit will the synthesis tool generate for the given designs ?

It will be a **shift register** (It is written in Non-blocking assignment)

Initial values

A=1, B=0; C=0; D=0; E=0;

→ NON-BLOCKING...

[7], A= 1, B= 0, C= 0, D= 0, E= 0
[15], A= 1, B= 1, C= 0, D= 0, E= 0
[25], A= 1, B= 1, C= 1, D= 0, E= 0
[35], A= 1, B= 1, C= 1, D= 1, E= 0
[45], A= 1, B= 1, C= 1, D= 1, E= 1
[55], A= 1, B= 1, C= 1, D= 1, E= 1

→ Dose not depend on the order of these statements

Ring counter: Using Non-Blocking Statement

```

module ring_cntr (clk, init, count);
    input clk, init;
    output reg [7:0] count;

    always @ (posedge clk)
    begin
        if (init)
            count = 8'b10000000;
        else
            begin
                count <= count << 1;
                count[0] <= count[7];
            end
        end
    end
endmodule

```

A ring counter

<https://www.edaplayground.com/x/cdbK>

```

module ring_counter_tb;
    reg clk, init;
    wire [7:0] count; integer i;

    ring_cntr rc(clk, init, count);
    always #5 clk = ~clk;

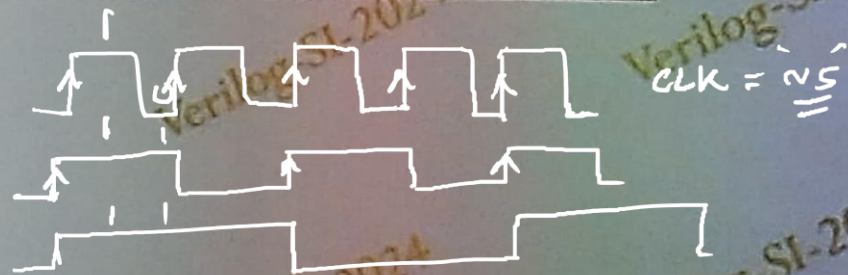
    initial begin
        clk <= 0; init <= 1;
        $monitor ("[%0t] init=%0b\n", $time, init, count);
        #7 init = 0;
        #50 $finish;
    end
endmodule

```

NOT ASSIGNED Anything !!

0]	init=1	count=xxxxxxxx
5]	init=1	count=10000000
7]	init=0	count=10000000
15]	init=0	count=00000001
25]	init=0	count=00000010
35]	init=0	count=00000100

clock started @ t=5
hence count = 8'b100



hence, 15 → 2
=

up + down = 5 + 5 = 10 seconds

NOT POSSIBLE
in Blocking.

only possible
via NON-BLOCKING
Assignment statements.

Ring counter: Using Blocking Statement (Wrong)

```
module ring_ctr (clk, init,
count);
input clk, init;
output reg [7:0] count;

always @ (posedge clk)
begin
if (init)
count = 8'b10000000;
else
begin
count = count << 1;
count[0] = count[7];
end
end
endmodule
```

A ring counter

```
module ring_counter_tb ;
reg clk, init;
wire [7:0] count; integer i;

ring_ctr rc(clk, init, count);
always #5 clk = ~clk;

initial begin
clk <= 0; init <= 1;
$monitor ("%0t init=%0b\n", $time, init, count);
#7 init = 0;
#50 $finish;
end
endmodule
```

```
[0] init=1 clk=0 count=xxxxxxxx
[5] init=1 clk=1 count=10000000
[7] init=0 clk=1 count=10000000
[10] init=0 clk=0 count=10000000
[15] init=0 clk=1 count=00000000
```

WRONG !!

- Since blocking assignment
- after first left shift count become 0

Counter: Async clr

```
module counter (mode, clr, clk,  
count);  
input mode, clr, clk;  
output reg [3:0] count;  
  
always @ (posedge clk, clr)  
if (clr) → RESET  
count <= 0; ✓  
else if (mode) → increment  
count <= count + 1; ✓  
else → decrement By 1  
count <= count - 1;  
endmodule
```

<https://www.edaplayground.com/x/9fys>

```
module counter_tb ;  
reg mode, clr, clk;  
wire [3:0] count;  
counter c1(mode, clr, clk, count);  
always #5 clk = ~clk;  
initial  
begin  
clk <= 0; clr <= 1;  
mode <= 1;  
fork  
#7 clr = 0;  
#82 clr = 1;  
#107 clr = 0;  
join  
#200 $finish;  
end  
endmodule
```

NOT ASSIGNED / Initialised

```
[0] mode=1 clr=1 count=xxxx  
[5] mode=1 clr=1 count=0000  
[7] mode=1 clr=0 count=0000  
[15] mode=1 clr=0 count=0001  
[25] mode=1 clr=0 count=0010  
[35] mode=1 clr=0 count=0011  
[45] mode=1 clr=0 count=0100  
[55] mode=1 clr=0 count=0101  
[65] mode=1 clr=0 count=0110
```